

ممتاز. بما إنك وافقت على Option A، هنبدأ التنفيذ فعليًا من هنا خطوة بخطوة، من غير ما نغيّر الـ docker-compose.yml الكبير.

القاعدة من الآن: أي قرار جديد يغير المنتج، الـ architecture، الـ schema،
الـصلاحيات، الـ UX flow أو الـ dependency أساسية → الـ agent يعرضه عليك الأول ولا
ينفذه. أما التفاصيل البصرية البحتة فمفوضة لـ Impeccable.

Step 0 — افتح المشروع من المكان الصحيح

حط محتويات آخر package في directory المشروع. المفروض يبقى عندك تقريبًا:

```
/dabber
AGENTS.md —|
CLAUDE.md —|
START_HERE.md —|
package.json —|
/docs —|
/supabase —|
/apps —|
/packages —|
```

ثم من نفس الـ terminal اللي هتشغل منه Claude Code أو Codex:

```
cd /path/to/dabber
```

```
node -v
npm -v
git --version
docker info
```

أهم command هنا:

```
docker info
```

لازم يشتغل من نفس البيئة اللي الـ agent هيشغل فيها.

لأن npx supabase start محتاج الوصول للـ Supabase CLI الـ Docker daemon. تشغل الـ local stack عن طريق (Supabase). Docker-compatible runtime.

Node

بسبب Impeccable، عايزين:

```
Node >= 22.18
```

Impeccable حاليًا تشترط Node.js 22.18 أو أحدث. (Impeccable Style)

لو:

```
node -v
```

طلع مثلاً:

```
v20.x
```

ما تبدأش لسه. حدّث Node على الـ host اللي عليه Codex/Claude.

وجود node:20 في الـ MCP container عندك مش مهم؛ إحنا بنتكلم عن الـ Node اللي الـ agent والـ project commands هيشغلوا بيه.

Step 1 — تثبيت dependencies

من root:

```
npm install
```

وبعدين:

```
npx supabase --version
```

إحنا مستخدمين Supabase CLI كـ project dev dependency، وبالتالي الصيغة الصحيحة هي:

```
... npx supabase
```

وده workflow رسمي مدعوم من (Supabase). Supabase.

Step 2 — تأكد إن Docker الحالي سليم

قبل Supabase:

```
docker ps
```

المفروض تشوف services الحالية زي:

```
azm-mysql8
```

```
azm-php82
```

```
azm-mailpit
```

```
azm-mcp
```

```
azm-n8n
```

ما تعملش stop لأي واحدة منهم.

وبالأخص ممنوع:

```
docker compose down
docker system prune
docker volume prune
```

Step 3 — شغل Claude Code أو Codex

دلوقتي افتح الagent من root المشروع نفسه.

Codex مثلاً:

```
codex
```

أو Claude Code:

```
claude
```

بعدها أول Prompt هتديله هو ده.

.We are starting the implementation of Dabber

Before executing anything, read the project documentation completely enough to
:understand the approved architecture and operating rules, especially

- AGENTS.md -
- CLAUDE.md if applicable -
- START_HERE.md -
- docs/DECISIONS.md -
- docs/PRD.md -
- docs/ARCHITECTURE.md -
- docs/DATABASE.md -
- docs/MIGRATIONS.md -
- docs/PERMISSIONS.md -
- docs/SUPER_ADMIN.md -
- docs/UI_DESIGN_GOVERNANCE.md -
- docs/LOCAL_ENVIRONMENT.md -
- /all ADRs under docs/adr -

:Approved decisions for this session

- .Package manager: npm only .1
- .Frontend/full-stack framework: Next.js + TypeScript .2
- .Database/Auth platform: Supabase .3
- .Hosting later: Vercel .4
- .apps/web and apps/admin must remain separate applications .5
- .The Super Admin application is completely separate from the household application .6
- :Local Supabase strategy is OPTION A .7

use `npx supabase start` to create an isolated Dabber Supabase stack on the existing
 .Docker daemon

- .Do NOT modify the host/shared docker-compose.yml .8
- Do NOT stop, restart, prune, remove or modify unrelated Docker containers, networks .9
- .or volumes
- .Database migrations and declarative schemas must remain version-controlled .10
- .npm must be used; do not introduce pnpm, yarn or bun .11
- .Impeccable is the delegated authority for visual/UI craft .12
- Product behavior, permissions, schema semantics, security, architecture and feature .13
- .scope are NOT delegated to Impeccable

:OWNER DECISION GATE

If you encounter a choice that materially affects any of the following, STOP before
 :implementation and ask me

- product behavior -
- database/domain model -
- schema semantics -
- RLS or authorization -
- privacy -
- financial calculations -
- authentication behavior -
- application architecture -
- deployment architecture -
- new major dependency -
- business workflow -
- feature scope -
- destructive migration -
- irreversible data operation -

:When asking, provide

the decision to make .1

your recommended option .2
alternatives .3
trade-offs .4
what is blocked until I decide .5

.Do not silently make that decision

You MAY independently make low-level reversible implementation choices that do not
.change those approved decisions

:UI / UX RULE

:Impeccable has authority over visual craft including

composition -
visual hierarchy -
typography -
spacing -
visual styling -
responsive presentation -
motion -
transitions -
micro-interactions -
loading states -
empty states -
visual feedback -
interface polish -

It does NOT have authority to alter product behavior, permissions, privacy, financial
.logic or data visibility

:TASK FOR THIS SESSION

.Perform ONLY environment/bootstrap validation and local Supabase initialization

:Verify .1
Node >= 22.18 -
npm -
Git -
Docker daemon access -

.Inspect existing project files and package.json .2

.Run npm install if required .3

:Verify the Supabase CLI with .4

`npx supabase --version`

.Check whether `supabase/config.toml` already exists .5

:If Supabase has not yet been initialized locally, run .6

`npx supabase init`

Do not overwrite existing Dabber migrations, schemas, seed files, tests or
documentation

.Review generated `supabase/config.toml` .7

The `project_id` must uniquely identify Dabber and must not collide with other local
Supabase projects

If changing the generated `project_id` requires a project-level naming decision that is not
already documented, ask me before changing it

Before starting Supabase, inspect currently bound Docker ports / running containers .8
enough to detect obvious conflicts

.Do not modify unrelated services to resolve conflicts

:Start the isolated Dabber local Supabase stack using .9

`npx supabase start`

:Run .10

`npx supabase status`

:Report the local .11

API URL -

Studio URL -

database URL/port -

email testing URL -

CLI version -

:SECURITY

Do not paste secret/service-role keys into committed files, documentation, chat
.summaries or source code

Local secret values may only be placed in ignored local environment files when they are
.actually required later

:IMPORTANT

:Do NOT yet

- link a Supabase Cloud project
- push anything remotely
- create a Vercel project
- scaffold or redesign the Next.js apps
- implement authentication
- implement product features
- modify the database model
- modify migrations to make them pass
- initialize Impeccable
- run migrations yet

This task ends when the local Supabase stack is running successfully and its status has
.been verified

:At the end, report

- commands actually executed
- pass/fail for each environment check
- files created or modified
- currently running Dabber Supabase containers
- local URLs
- warnings or blockers

.Never claim a command succeeded unless you actually ran it

ليه موقفين ال-Prompt هنا؟
متعمد.

مش عايز Codex يعمل:

init → migration → Next.js → Auth → UI

مرة واحدة.

عائزين الأول نعرف إن:

Existing Docker projects
untouched ✓

Dabber Supabase
isolated ✓

Supabase Studio
opens ✓

Local API
works ✓

وبعدئ ننتحر.

Step 4 — بعد نجاح Prompt 01

المفروض npx supabase status يرجع حاجة شبيهة بـ:

```
API URL: http://127.0.0.1:54321
...//:DB URL: postgresql
Studio URL: http://127.0.0.1:54323
```

Supabase init تنشئ supabase/config.toml و start تطلع الـ local Supabase services.
(Supabase)

ساعتها افتح:

```
http://localhost:54323
```

وتأكد إن Supabase Studio شغالة.

Step 5 — Impeccable لسه لأ

متعمد إن Prompt 01 يقول:

```
.Do not initialize Impeccable yet
```

الأفضل نعمل:

Infrastructure
↓
Database baseline
↓
Next.js scaffold
↓
Impeccable init
↓
Design system
↓
Features

لأن impeccable init يقرأ المشروع ويحفظ فهم المنتج والجمهور والأهداف في PRODUCT.md. لو عملناه قبل وجود الapps والstructure الأساسية، السياق المتاح له سيكون أقل. ([Impeccable Style](#))

لما نوصل له، التثبيت الحالي رسميًا:

```
npx impeccable install
```

وبعدها Codex:

```
impeccable init$
```

Claude Code:

```
impeccable init/
```

([Impeccable Style](#))

نقطة مهمة عن استخدام Claude وCodex بالتبادل

ينفع جدًا تعمل:

Codex
↓
يعمل Database task

Claude Code
↓
يعمل UI task

Codex
↓
يراجع business logic

Claude



يعمل Impeccable polish

لكن مش في نفس اللحظة على نفس working tree.
الأفضل:

`git status`

قبل تبديل الـ agent.

وعند اكتمال milestone:

`git add` .
`"git commit -m 'chore: bootstrap local supabase'"`

بعدها بدّل Agent.

ده هيخلينا نعرف بالضبط مين غير إيه ونقدر نرجع لأي نقطة بسهولة.

المطلوب منك دلوقتي فقط

نفذ أول 4 commands:

`node -v`

`npm -v`

`git --version`

`docker info`

لو كلهم تمام، اعمل:

`npm install`

ثم افتح **Codex** أو **Claude Code** وابعث له **Prompt 01 أعلاه**.

بعد ما يخلص، ابعثلي الـ **report النهائي بتاعه كما هو**. الخطوة اللي بعدها هتكون محددة جدًا: **تشغيل الـ baseline migrations واختبار RLS لأول مرة**، ولن نغير أي migration أو schema decision بدون ما نراجع مع بعض.